Re-executing Scientific Design after Method Changes

Public development manuscript skeleton, 19 October 2026

Working draft

Abstract

Changing a device, changing the method that designs it, and changing the procedure that improves that method invalidate different computations. We study these three interventions separately in an executable battery-free design apparatus. This development draft contains deterministic invalidation controls and a preregistered four-attempt prompt intervention. Actual paid results are inserted only from sealed receipts. No recursive self-improvement, unseen-task transfer, hardware validation, or speedup is established by the present draft.

1 Question and scope

A previously successful device trace can cease to describe the system after a legal change. Reusing that trace may silently preserve obsolete control decisions. At the method level, a changed global design prompt may alter the first observation, the candidate, and every later experiment. Evaluating the old device under that prompt does not evaluate the changed method. Our question is which results remain valid after a typed intervention, and how to confirm a proposed method change without charging a cached response as a new design attempt.

2 Objects and validity contract

Let D contain device structure, circuits, firmware, storage, communication, and deployment. Let H contain the design prompt, skills, available observations, context policy, experiment order, and organization. Let G propose and select changes to H. Device runtime, the agent's decision span, and method generations are separate quantities. A record is reusable only when all consumed inputs, environment identities, numerical settings, and output hashes remain applicable. An evaluation change may reuse raw physical output while requiring a new verdict.

For a global prompt change $H \to H'$, our conservative rule restarts at task entry with a new owner, initial candidate, and context. Exact lower-level input responses may still be reused. A local continuation requires a separate proof that its restored state is valid. A G comparison additionally fixes the starting H_0, development distribution, selection protocol, and improvement budget.

3 F1a: device-response controls

We compare old fixed-trajectory replay, conservative dependency invalidation, boundary invalidation, full response execution, and exact-input response caching. The first qualification apparatus has energy arrivals, finite storage, a threshold controller, and load consumption. The independent full response is the oracle. In a feedback case, both dependency and boundary invalidation expand to the full response; this null advantage is a required control, not a speedup result.

In the executed deterministic controls, fixed-trajectory replay disagrees with the full response on 5 of 6 inputs. The conservative dependency, boundary, and exact-cache results agree on all 6. Only the unchanged input is an initial exact-cache hit. These are synthetic correctness controls; execution counts do not measure wall-time speedup.

Input	Replay packets	Full packets	Replay valid
unchanged	4	4	True
less-energy	4	2	False
higher-load	4	3	False
threshold-only	4	3	True
capacity-limit	4	0	False
arrival-order	4	2	False

Paid confirmation status. No paid result is available in this data snapshot.

4 F1b and early F4: real method re-execution

The frozen public task uses the already qualified piezoelectric, vendor PMIC, host-C controller, and digital receiver apparatus. The power board is fixed; the agent must revise firmware and deliver six valid frames in twenty seconds. The declared initial charge and current envelope remain assumptions. The task is an exposed development derivative of historical T1, not a fresh holdout.

H_0 requests one complete design attempt with execution, validation, decision, report, release, and close. H_1 adds a constraint and time-origin audit before a single justified revision, with version-bound evidence. Both receive the same full task and sources, backend, tool definitions, limits, and initial native cache. The prompt update was proposed by the research operator from earlier timing and tool-use failures; it is not an autonomous improver. Two attempts per condition run in order H_0, H_1, H_1, H_0. Each allows at most 80 parent-plus-child requests, four candidate evaluations, and 1,200 seconds of decision wall time. Organization is single-agent. No automatic resampling is allowed.

Full execution confirms every attempted method. Development retention requires both H_1 confirmations to deliver admissible results without an increased failure count. With two attempts per condition, cost and quality differences are descriptive. A tie, failure, or no retained update is a valid result.

5 Costs, exposure, and competing explanations

Report all parent and child requests, settled usage, reservations, failed tools, native dispatches, retries, confirmations, cache construction, and unresolved UNKNOWN separately. Total cost is

$$C = C_{\text{proposal}} + C_{\text{selection}} + C_{\text{confirmation}} + C_{\text{failures}} + C_{\text{cache construction}}.$$

A valid cache hit does not remove the cost of building its entry. Paired attempts start from identical cache manifests rather than warming later conditions with earlier outputs. Provider identity and usage availability are dispatch gates. Preflight failures remain recorded, and cannot become successful attempts by silent replacement. Mechanism exposure is shared with the interface study.

F1a supports a correctness boundary for this small control; it does not establish a general incremental algorithm. F1b can establish that a prompt was actually changed and new design decisions executed. Favorable development outcomes would still admit task familiarity, stochastic variation, and operator selection as competing explanations. Later selection and final tasks must be frozen separately.

6 Related work and remaining experiments

Executable build-system decompositions provide a strong dependency-invalidation baseline [1]. Prompt optimizers such as GEPA propose and evaluate prompt changes using execution feedback [2]. Our current work does not establish superiority over either family. The planned study isolates physical-response reuse from full method confirmation and retains negative cases. F2/F3 will explain decision spans and organizations on a useful subset. F5 requires independent G_0/G_1 lineages from a fixed H_0; no such lineage experiment is reported. F6 will follow the continuous A/R product after its service reference is qualified. Physical calibration and hardware acceptance remain separate work.

References

[1] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems a la carte. ICFP, 2018. https://www.microsoft.com/en-us/research/publication/build-systems-la-carte/.

[2] Lakshya A. Agrawal et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. arXiv:2507.19457v2, 2026. https://arxiv.org/abs/2507.19457v2.